



9.3. Converting Infix Expressions using Stacks

Previous Section:

 [9.2. Expression Evaluation using Stacks](#)

Contents:

[1. Infix to Postfix Conversion](#)

[2. Infix to Prefix Conversion](#)

[Summary](#)

[References](#)

Here, we will reuse the Stack object implemented in the previous section:

[Stack.py](#)

In the previous section, we learned how postfix and prefix expressions can be evaluated efficiently using stacks. However, most users naturally write expressions in **infix notation**.

For example: `1 + 2 * 3` or `(1 + 2) * 3 - 5`

Since infix expressions contain operator precedence and parentheses, computers often convert them into postfix or prefix notation before evaluation.

In this section, we will learn how stacks help us convert infix to postfix and prefix

1. Infix to Postfix Conversion

To convert an infix expression into a postfix expression, we scan the expression **from left to right**. During the process, we use:

- A stack to temporarily store operators and parentheses.
- A postfix list to store the final result.

Conversion Rules - For each entry in the infix expression:

- **Rule 1: Opening Parenthesis.** If the entry is `(`, push it onto the stack.

Stack

Top

↓

(

- **Rule 2: Operand.** If the entry is a number or variable like `1, 2, a, b`, append it directly to the postfix expression.

Postfix

1 2

- **Rule 3: Operator.** If the entry is an operator like `+, -, *, /`, we compare its precedence with the operator currently at the top of the stack.
 - Case A: Top is not an operator. Push the new operator.

Stack

Top

↓

+

| (|

- Case B: Top operator has lower precedence. Push the new operator.
Example:

```
Top = +  
  
Current = *  
  
* has higher precedence  
  
Therefore: Push *
```

- Case C: Top operator has higher or equal precedence. Pop operators from the stack and append them to the postfix expression until the stack becomes empty, or an operator with lower precedence is found.
- Rule 4: Closing Parenthesis. If the entry is `)`, pop operators from the stack and append them to the postfix expression until the matching opening parenthesis is found. The opening parenthesis itself is discarded.

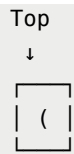
Example: Convert `(1 + 2) * 3 - 5`

- Initial State

```
Infix:  
  
( 1 + 2 ) * 3 - 5  
  
Postfix: (empty)  
  
Stack:  
  
(empty)
```

- **Step 1:** Push `(` to stack.

Stack

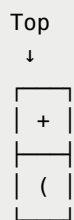


- Step 2: Encounter **1**, an operand → Append to postfix.

Postfix: 1

- Step 3: Push **+**.

Stack



- Step 4: Encounter **2** → Append to postfix.

Postfix: 1 2

- Step 5: Encounter **)** → Pop until "**(**". Meaning pop **+** → Append to postfix.

Postfix: 1 2 +

Remove "(".

Stack

(empty)

- Step 6: Push ***** to stack.

Stack

Top

↓

*

- Step 7: Encounter **3** → Append to postfix.

Postfix: 1 2 + 3

- Step 8: Encounter **-**. But since current top is *****, which has higher precedence than **-**, → Pop *****. Push **-** **to the stack**.

Postfix: 1 2 + 3 *

Stack

Top

↓

-

- **Step 9:** Encounter **5** → Append to postfix.

Postfix: 1 2 + 3 * 5

- **Step 10:** Expression complete. Pop remaining operators (which is only **-**) → Append to postfix.

Postfix: 1 2 + 3 * 5 -

- **Final Result**

(1 + 2) * 3 - 5

↓

1 2 + 3 * 5 -

Operator Precedence Function

```
def get_precedence(operator):  
    precedence = {"+": 1, "-": 1, "*": 2, "/": 2}  
    return precedence.get(operator, 0)
```

Infix to Postfix Implementation:

```
def infix_to_postfix(infix_list):  
    operators = ["+", "-", "*", "/"]  
    stack = Stack()  
    postfix = []  
  
    for entry in infix_list:  
        if entry == "(":  
            stack.push(entry)  
        elif entry == ")":  
            entry2 = stack.pop()  
            while entry2 != "(":  
                postfix.append(entry2)  
                entry2 = stack.pop()  
  
        elif entry in operators:  
            if not stack.is_empty():
```

```

        precedence = get_precedence(entry)
        entry2 = stack.pop()
        while precedence <= get_precedence(entry2):
            postfix.append(entry2)
            if not stack.is_empty():
                entry2 = stack.pop()
            else:
                break
        if precedence > get_precedence(entry2):
            stack.push(entry2)
        stack.push(entry)
    else:
        postfix.append(entry)

    while not stack.is_empty():
        postfix.append(stack.pop())

    return postfix

```

2. Infix to Prefix Conversion

Prefix conversion follows a very similar idea, but there is one additional step. Instead of scanning the original infix expression, we first reverse it.

General Algorithm

- **Step 1:** Reverse the infix expression. Example: `(1 + 2 * 3) - 5` becomes `5 - ) 3 * 2 + 1 (`
- **Step 2:** Scan from left to right. Use rules similar to infix-to-postfix conversion. The difference is: **Opening and closing parentheses switch roles.**
- **Step 3:** Build an intermediate postfix expression.
- **Step 4:** Reverse the postfix expression. The result becomes the prefix expression.

Example: Convert `(1 + 2 * 3) - 5` → Reverse Expression `5 - ) 3 * 2 + 1 (`

- Step 1: Append 5 to output

Output: 5

- Step 2: Push `-` to stack

Stack

Top



-

- Step 3: Push `)` to stack

Stack

Top



)
-

- Step 4: Append 3

Output: 5 3

- Step 5: Push `*`

Stack

Top



*
)



- Step 6: Append 2

Output: 5 3 2

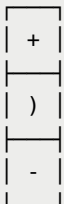
- Step 7: Encounter $+$. But since the current top is $*$, which has higher precedence, pop $*$ and push $+$

Output: 5 3 2 *

Stack

Top

↓



- Step 8: Append 1

Output: 5 3 2 * 1

- Step 9: Encounter $($ → Pop until $)$. Pop $+$ from the stack and append it to the output → Remove the matching $)$.

Output: 5 3 2 * 1 +

- Step 10: Pop remaining operators, which is only $-$ → Expression complete.

Output

5 3 2 * 1 + -

- Step 11: Reverse the output 5 3 2 * 1 + - to:

- + 1 * 2 3 5

- Final Result

(1 + 2 * 3) - 5

↓

- + 1 * 2 3 5

Infix to Prefix Implementation

```
def infix_to_prefix(infix_list):
    operators = ["+", "-", "*", "/"]
    stack = Stack()
    prefix = []

    for entry in reversed(infix_list):
        if entry == ")":
            stack.push(entry)
        elif entry == "(":
            entry2 = stack.pop()
            while entry2 != ")":
                prefix.append(entry2)
                entry2 = stack.pop()

        elif entry in operators:
            if not stack.is_empty():
```

```

        precedence = get_precedence(entry)
        entry2 = stack.pop()
        while precedence < get_precedence(entry2):
            prefix.append(entry2)
            if not stack.is_empty():
                entry2 = stack.pop()
            else:
                break
        stack.push(entry2)
        stack.push(entry)

    else:
        prefix.append(entry)

while not stack.is_empty():
    prefix.append(stack.pop())

return reversed(prefix)

```

Summary

Here we will summarize **section 9.2. and 9.3.** altogether:

Expression evaluation is one of the most important applications of stacks because it demonstrates how stacks can manage operators, operands, and precedence rules efficiently.

By using stacks, we can:

- Evaluate postfix expressions from left to right.
- Evaluate prefix expressions from right to left.
- Convert infix expressions into postfix notation.
- Convert infix expressions into prefix notation.

The stack temporarily stores operators and parentheses, ensuring that operations are performed in the correct order. This makes stacks an essential component of calculators, compilers, interpreters, expression parsers, and many other software systems that process mathematical or logical expressions.

References

https://drive.google.com/file/d/1yBJEh1C9V-nXZ7TF80JtWvozfaDYRq5B/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/convert-infix-expression-to-postfix-expression/>

<https://www.geeksforgeeks.org/dsa/convert-infix-prefix-notation/>

Next Section:

 [10. Queues](#)